

Trabajo Especial de Modelos y Simulación: Tema C

Ignacio Benjamín Ceballos Tomas Luciano Loyber

Junio 2025

Resumen

El objetivo principal de este proyecto es evaluar y comparar el comportamiento y la calidad de tres generadores de números pseudoaleatorios de distintas familias y complejidades, aplicados a una simulación de un sistema real. Para llevar a cabo la comparación, se seleccionaron tres generadores

- Generador congruencial lineal
- XORshift
- Xoshiro

Para evaluar el desempeño de los generadores, se simulara un sistema de colas con un único servidor. El tiempo de arribo de los clientes sigue un proceso de Poisson no homogéneo con intensidad:

$$\lambda(t) = 30 + 30 \cdot \sin\left(\frac{2\pi t}{24}\right)$$

Y los tiempos de atención siguen una distribución exponencial $\mu = 40$.

El objetivo de la simulación es observar cómo evolucionan la longitud de la cola, la utilización del servidor, entre otras métricas, a lo largo de un período de 48 horas

1 Descripción teórica de los generadores

1.1 Generador congruencial lineal

1.1.1 Fórmula y parámetros

El Generador de Congruencia Lineal, es un algoritmo que permite obtener una secuencia de números pseudoaleatorio.

Sean a, c, M, y_0 enteros no negativos, $M \geq 2$, la formula para generar la secuencia es:

$$y_i = ay_{i-1} + c \mod M, \quad i \geq 1,$$

La secuencia en $[0, 1)$ se obtiene dividiendo los valores y_i por M :

$$x_i = \frac{y_i}{M}.$$

La secuencia generada es determinista y depende de los valor que se le asignen a:

- y_0 : semilla
- a : multiplicador
- c : incremento
- M : módulo

Para este análisis usamos $a = 16807, c = 0, M = 2^{31} - 1$. Por lo tanto, como $c = 0$, estudiaremos la versión del generador congruencial lineal multiplicativo.

1.1.2 Calidad esperada

Cuando el generador es multiplicativo, su período máximo es de $M - 1$ si se cumple que: M es primo y a es una raíz primitiva módulo M . Como esto se cumple para los parametros que usamos, el período máximo es:

$$2^{31} - 2$$

Esto nos dice que se repetirán valores recién luego de $2^{31} - 2$ iteraciones; esa es la longitud de la secuencia. Como no se repite hasta haber generado mas de 2 mil millones de valores, es un periodo bueno.

Su ventaja mas destacable es la simplicidad[2]; el generador presenta una buena eficiencia computacional debido a la simplicidad de su fórmula, basada únicamente en una multiplicación y una operación módulo. Sin embargo, se afirma que la aleatoriedad no es de buena calidad para cierto tipo de simulaciones que requieren generación de puntos en espacios multidimensionales, como por ejemplo una simulación de Monte Carlo.

- Periodo: $2^{31} - 2$ (para la versión implementada)
- Velocidad: Muy rápida
- Calidad: Media para simulaciones básicas y mala en pruebas multidimensionales

1.2 XORshift

1.2.1 Fórmula y parámetros

La forma general del Xorshift (Marsaglia, 2003)[1]: Sean $a, b, c, x_0 \in \mathbb{Z}$

$$x_i \leftarrow x_{i-1} \oplus (x_{i-1} \ll a)$$

$$x_i \leftarrow x_i \oplus (x_i \gg b)$$

$$x_i \leftarrow x_i \oplus (x_i \ll c)$$

Se usan tres operaciones shift y tres o cuatro exclusive-or; a , b y c denotan la cantidad de bits que serán desplazados en cada paso. Existen muchas versiones dependiendo la cantidad de bits con la que se quiera generar cada número. En este trabajo usamos la versión clásica de 32 bits, con $a = 13, b = 17, c = 5$.

En la implementación en Python, como los enteros no tienen un límite fijo de 32 bits, es necesario aplicar una máscara binaria con 0xFFFFFFFF (equivalente a $2^{32} - 1$) después de los desplazamientos a la izquierda. Esto asegura que los valores se mantengan dentro del rango de 32 bits sin signo, como en el diseño original del algoritmo.

Sea x_i es el entero generado por XORshift y n el número de bits del generador, La secuencia en $[0, 1)$ se obtiene calculando:

$$u_i = \frac{x_i}{2^n}.$$

1.2.2 Calidad esperada

Su principal ventaja es la alta eficiencia computacional, XORshift logra obtener un alto periodo para sus secuencias, utilizando pocos recursos. El periodo de la versión que usamos es de $2^{32} - 1$; Además, es más rápido que GCL pues solo usa la operación XOR y desplazamientos de bits, que son muy rápidos en hardware y software moderno, mientras que las multiplicaciones, como usa GCL, suelen ser más lentas. Esto hace a XORshift ideal para aplicaciones que requieren generar muchos números rápidamente. En cuanto a la calidad estadística, es mejor que GCL pero tiene ciertas debilidades, probadas utilizando test suites.

- Periodo: $2^{31} - 1$ (para la versión de 32b)
- Velocidad: Extremadamente rápida, ya que son operaciones solo con bits
- Calidad: Buena para la mayoría de las aplicaciones

1.3 Xoshiro

1.3.1 Fórmula y parámetros

Esta es una versión modificada de Xorshift que agrega rotaciones. Puesto que cada versión de Xoshiro difiere considerablemente, damos directamente la formula de la versión Xoshiro256** (la que usamos) opera con numeros de 64 bits y mantiene un estado interno compuesto por $s_{0,0}, s_{1,0}, s_{2,0}, s_{3,0} \in \mathbb{Z}$.

La salida del generador se calcula mediante una combinación de multiplicaciones y una rotación

$$\text{output}_i = \text{rotl}(s_{1,i} \cdot 5, 7) \cdot 9$$

donde $\text{rotl}(x, k)$ representa una rotación a la izquierda de x en k bits. La actualización del estado interno es la siguiente:

$$\begin{aligned} t &\leftarrow s_{1,i} \ll 17 \\ s_{2,i+1} &\leftarrow s_{2,i} \oplus s_{0,i} \\ s_{3,i+1} &\leftarrow s_{3,i} \oplus s_{1,i} \\ s_{1,i+1} &\leftarrow s_{1,i} \oplus s_{2,i+1} \\ s_{0,i+1} &\leftarrow s_{0,i} \oplus s_{3,i+1} \\ s_{2,i+1} &\leftarrow s_{2,i+1} \oplus t \\ s_{3,i+1} &\leftarrow \text{rotl}(s_{3,i+1}, 45) \end{aligned}$$

Ahora el estado interno fue modificado y $s'_{0,0}, s'_{1,0}, s'_{2,0}, s'_{3,0}$ se usaran para la siguiente llamada al generador.

Este generador se inicializa con una semilla de 64 bits a partir de la cual se construyen los 4 numeros del estado utilizando el generador auxiliar SplitMix64, el cual garantiza buena dispersión inicial.

Para que la secuencia este en $[0,1)$ se calcula

$$u_i = \frac{\text{output}_i}{2^{64}}$$

1.3.2 Calidad esperada

La calidad de xoshiro es excelente. El punto más fuerte de Xoshiro es su capacidad para pasar unas de las baterías de pruebas estadísticas más exigentes que existen, *TestU01* y *BigCrush*. Ademas tienen un periodo muy largo, $2^{256} - 1$, por lo que es prácticamente imposible que la secuencia se repita. Tambien, al igual que Xorshift cuenta con una velocidad extremadamente rápida debido que solo se usan operaciones ente bits

- Periodo: $2^{256} - 1$
- Velocidad: Extremadamente rápida, ya que son operaciones solo con bits
- Calidad: Excelente

2 Descripción del problema

2.1 Problema a simular

Como ya fue descrito en el resumen, se desea simular un sistema de colas con un único servidor.

- El tiempo de arribo de los clientes sigue un proceso de Poisson no homogéneo con intensidad: $\lambda(t) = 30 + 30 \cdot \sin(\frac{2\pi t}{24})$
- Los tiempos de atención siguen una distribución exponencial $\mu = 40$.

2.2 Tiempos de atención y Arribo

Para simular el proceso de Poisson no homogéneo de los tiempos de arribo utilizamos el método de adelgazamiento simple, generamos un proceso de Poisson homogéneo de tasa $\lambda_{max} = 60$ y usando aceptación y rechazo generamos el no homogéneo. Entonces cada arribo se cuenta con probabilidad $\frac{\lambda(t)}{\lambda_{max}}$.

Para generar variables aleatorias con distribución exponencial usamos el método de la transformada inversa. Y usamos los generadores para las v.a uniformes que requieren estos algoritmos.

2.3 Simulación del sistema de colas de un servidor

Antes de simular la atención, se generan todos los arribos, una lista con los tiempos de llegada de los clientes durante 48 horas; además se inicializan ciertas métricas para el posterior análisis del rendimiento de los generadores en los distintos aspectos. Para cada arribo, si el servidor está libre, atiende de inmediato, de lo contrario encontramos un cliente que estuvo en espera. Así que cuando llega un cliente que estuvo esperando lo contamos con la variable `cola_en_espera`.

```
if arribo >= tiempo_servidor:
    inicio_servicio = arribo
else:
    inicio_servicio = tiempo_servidor
    cola_en_espera += 1
```

Luego, se actualiza cada métrica: cola por hora, tiempos de espera, servicio, salida, uso del servidor por hora, y se guardan en listas que almacenen por hora.

```
espera = max(0, inicio_servicio - arribo)
servicio = generador_atencion(generador)
salida = inicio_servicio + servicio
for h in range(int(inicio_servicio), min(int(salida) + 1, horas)):
    uso_por_hora[h] += min(salida, h + 1) - max(inicio_servicio, h)
tiempos_espera.append(espera)
tiempos_en_sistema.append(salida - arribo)
tiempo_servidor = salida
```

3 Metodología

3.1 Test Realizados

Con el objetivo de realizar una comparación objetiva entre los distintos generadores, se ejecutaron múltiples simulaciones del mismo sistema de colas. Para tomar distintas métricas como:

- Tasa de utilización del servidor en función del tiempo.
- El tiempo promedio en el sistema por cliente.
- La distribución de los tiempos de espera.
- El porcentaje de tiempo que el servidor está ocupado.
- Evolución de la longitud de la cola en el tiempo.
- Histograma de los tiempos de espera en el sistema.
- Distribución del tiempo entre arribos y de servicios simulados

Cada simulación se hizo manteniendo constantes todos los parámetros involucrados. Los generadores se inicializaron todos con la misma *seed* = 1234567. Esto asegura que cualquier diferencia observada en los resultados se deba exclusivamente al comportamiento interno de cada generador y no a variaciones de otro tipo.

3.2 Implementación del experimento

Luego de implementar los generadores, implementamos las funciones para la generación de los tiempos de arribo y de atención:

```
def lambda_t(t):  
    return 30 + 30 * math.sin(2 * math.pi * t / 24)  
  
def generar_arribos(T, generador, lambda_max=60):  
    eventos = []  
    x = generador.random()  
    t = -math.log(x) / lambda_max  
    while t <= T:  
        v = generador.random()  
        if v < lambda_t(t) / lambda_max:  
            eventos.append(t)  
            t += -math.log(generador.random()) / lambda_max  
  
    return eventos  
  
def generador_atencion(generador, lambda_=40):  
    x = generador.random()  
    return -math.log(1-x)/lambda_
```

Para probar su funcionamiento, realizamos una versión preliminar de la simulación. La cual era un procedimiento que al llamarlo imprimía en pantalla: tiempo de arribo, tiempo de inicio de servicio, tiempo de servicio y tiempo de salida del servicio, para los cliente 1, 500, 1000, 1500.

```
def simular(generator, Horas = 48):
    arribos = generar_arribos(Horas, generator)
    print(f"Cantidad de arribos: {len(arribos)}")
    tiempo_servidor = 0
    cola_por_hora = [0] * Horas
    i = 0
    print(f"Simulación con generator: {generator.__class__.__name__}")

    for arribo in arribos:
        i += 1
        t_arribo = int(arribo)

        if i in [1, 500, 1000, 1500]:
            print(f"Arribo {i}: {arribo:.4f}")
        if arribo >= tiempo_servidor:
            tiempo_inicio = arribo
            if i in [1, 500, 1000, 1500]:
                print(f"Tiempo de inicio de servicio: {tiempo_inicio:.4f}")
        else:
            tiempo_inicio = tiempo_servidor
            cola_por_hora[t_arribo] += 1
            if i in [1, 500, 1000, 1500]:
                print(f"Tiempo de inicio de servicio (con cola): {tiempo_inicio:.4f}")

        servicio = generator_atencion(generator)
        if i in [1, 500, 1000, 1500]:
            print(f"Tiempo de servicio: {servicio:.4f}")
        tiempo_salida = tiempo_inicio + servicio
        if i in [1, 500, 1000, 1500]:
            print(f"Tiempo de salida de servicio: {tiempo_salida:.4f}")
        tiempo_servidor = tiempo_salida
        if i in [1, 500, 1000, 1500]:
            print(f"El cliente {i} fue atendido en la hora {int(tiempo_servidor)}")
        if i in [1, 500, 1000, 1500]: print("\n")
```

Para el momento de realizar las comparaciones de los generadores y hacer los gráficos, no dimos cuenta que necesitábamos diversos parámetros. La primera idea fue hacer una función de simulación por gráfico que devuelva las métricas que necesitábamos. Pero al intentar hacerlo nos dimos cuenta que nuestro código crecía muy rápido y había mucha repetición. Por lo tanto, decidimos hacer una única función de simulación que devuelva muchas métricas y al momento de usar la función, elegir las métricas necesarias para el momento.

```
def simular(generator, horas=48):
    arribos = generar_arribos(horas, generator)
```



```

tiempo_servidor = 0
cola_en_espera = 0
cola_por_hora = [0] * horas
tiempos_espera = []
tiempos_en_sistema = []
uso_por_hora = [0.0] * horas
tiempos_entre_arribos = [arribos[i+1] - arribos[i] for i in range(len(arribos)-1)]

for arribo in arribos:
    hora = int(arribo)

    if arribo >= tiempo_servidor:
        inicio_servicio = arribo
    else:
        inicio_servicio = tiempo_servidor
        cola_en_espera += 1

    cola_por_hora[hora] = cola_en_espera

    espera = max(0, inicio_servicio - arribo)
    servicio = generador_atencion(generador)
    salida = inicio_servicio + servicio

    for h in range(int(inicio_servicio), min(int(salida) + 1, horas)):
        uso_por_hora[h] += min(salida, h + 1) - max(inicio_servicio, h)

    tiempos_espera.append(espera)
    tiempos_en_sistema.append(salida - arribo)
    tiempo_servidor = salida

h = 0
r = 0
for h in range(horas,):
    if cola_por_hora[h] == 0 and cola_por_hora[h-1] != 0:
        r += cola_por_hora[h-1]
    cola_por_hora[h] = max(cola_por_hora[h] - r, 0)
    h += 1

clientes = len(arribos)
uso_por_hora = [min(1.0, u) for u in uso_por_hora]
prom_espera = sum(tiempos_espera) / clientes if clientes else 0
prom_en_sistema = sum(tiempos_en_sistema) / clientes if clientes else 0
tiempos_servicio = [t_en_sistema - t_espera for t_en_sistema, t_espera in zip(tiempos_en_sistema, tiempos_espera)]

return {
    "arribos": arribos,
    "esperas": tiempos_espera,
    "en_sistema": tiempos_en_sistema,

```

```
"tiempos_entre_arribos": tiempos_entre_arribos,  
"cola_por_hora": cola_por_hora,  
"uso_por_hora": uso_por_hora,  
"uso_total": sum(uso_por_hora),  
"clientes": clientes,  
"prom_espera": prom_espera,  
"prom_en_sistema": prom_en_sistema,  
"tiempos_servicio": tiempos_servicio  
}
```

4 Resultados

Para llevar a cabo las comparaciones de los generadores, realizamos diversas tomas de métricas y con ellas hicimos gráficos para apreciar las diferencias. A forma de ayuda con algunas comparaciones, hicimos el gráfico de $\lambda(t)$

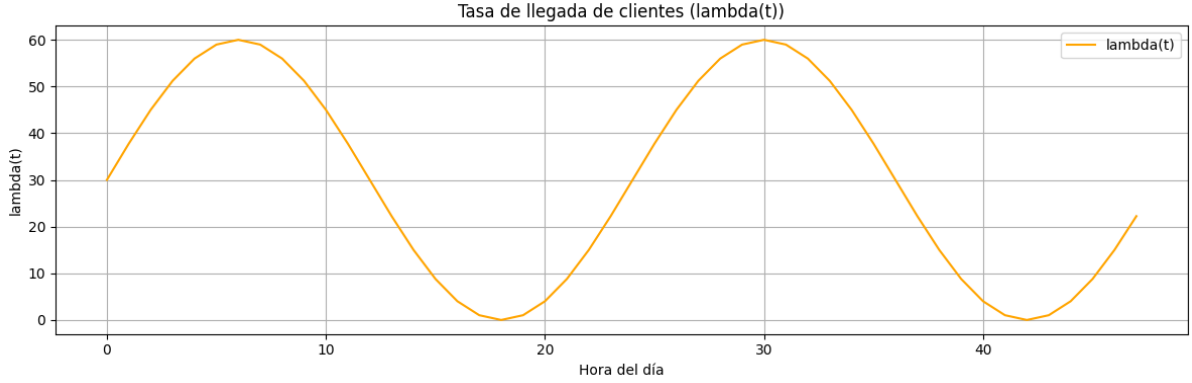


Figure 1: $\lambda(t)$

que es la función de intensidad de arribos de clientes al servidor.

4.1 Tasa de utilización del servidor en función del tiempo

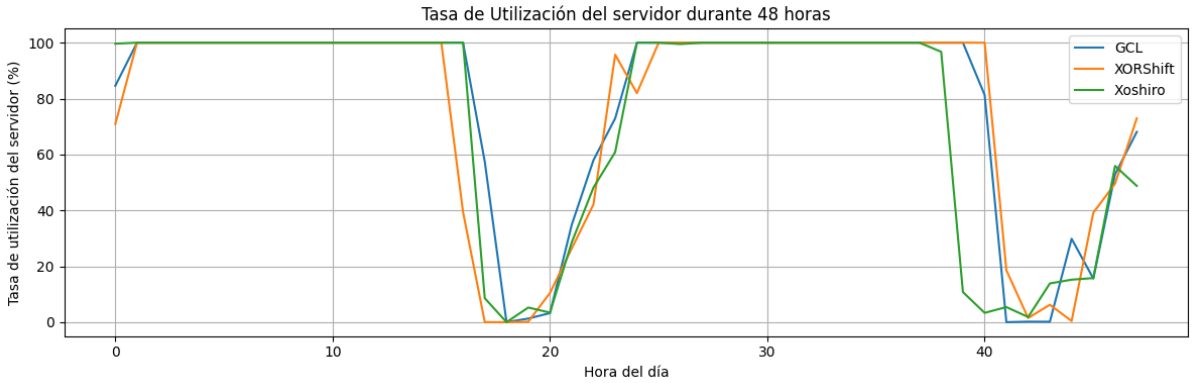


Figure 2: Tasa de utilización del servidor por hora

El gráfico muestra cómo varía la tasa de utilización del servidor durante las 48 horas simuladas para cada uno de los tres generadores. Se observa claramente un patrón: el servidor alcanza una utilización del 100% durante dos franjas horarias, y disminuye considerablemente el uso cerca de las 20hs y 40hs, lo cual es coherente con la función $\lambda(t)$ utilizada para modelar los arribos. Esta variación evidencia que el sistema responde correctamente a la distribución del flujo de clientes.

4.2 El tiempo promedio en el sistema por cliente

Antes de ver el gráfico debemos saber que el tiempo en el sistema de un cliente esta compuesto por $Tiempo\ en\ sistema = Tiempo\ en\ cola + Tiempo\ de\ uso$

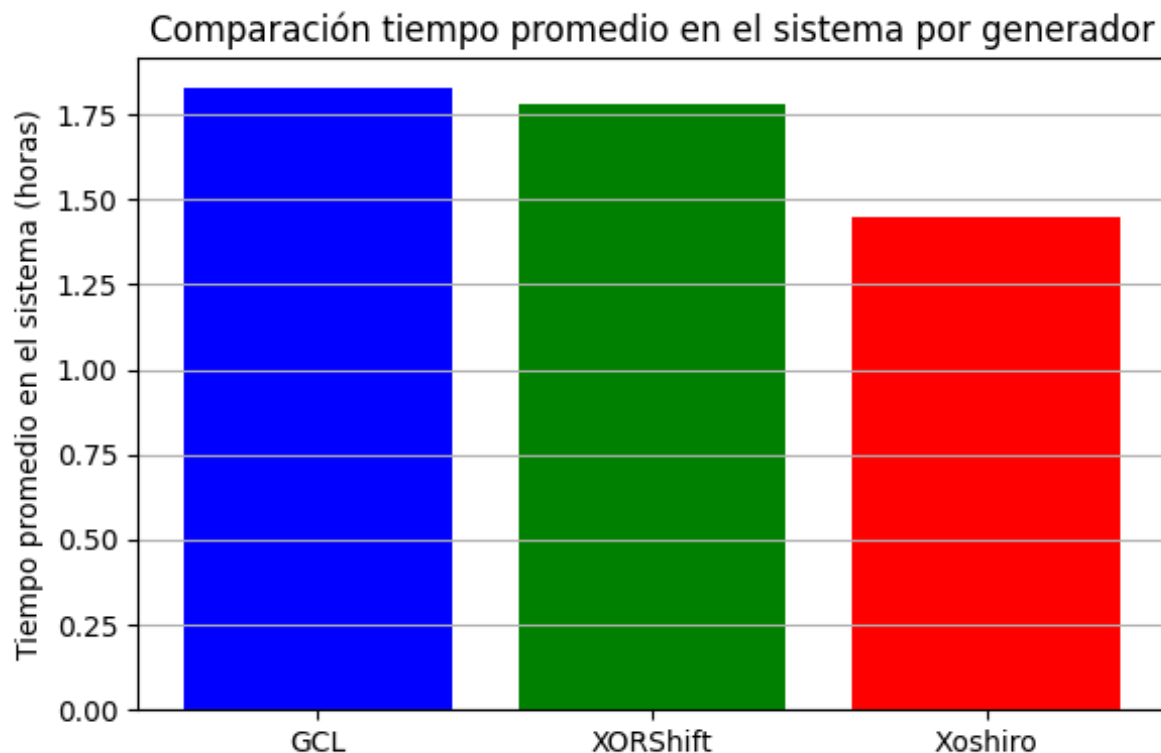


Figure 3: Tiempos promedios del cliente en el sistema

En este gráfico podemos ver que el generador GCL produce el mayor tiempo promedio en el sistema, seguido por XORShift, mientras que Xoshiro muestra un desempeño claramente superior, con un tiempo promedio significativamente más bajo que los otros dos. Esto da a entender que las secuencias pseudoaleatorias generadas por Xoshiro distribuyen mejor los arribos y tiempos de atención, evitando acumulaciones en la cola y reduciendo el tiempo total que un cliente permanece en el sistema.

4.3 La distribución de los tiempos de espera

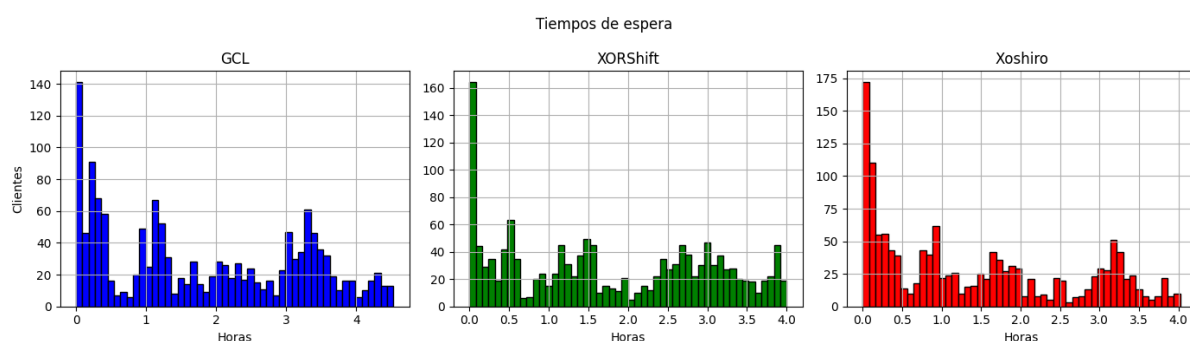


Figure 4: Distribución de tiempos de espera

En este conjunto de histogramas se representa la distribución de los tiempos de espera en el sistema para los tres generadores. Podemos observar que en los tres casos, la mayor

concentración de clientes tiene tiempos de espera muy bajos (entre 0 y 0.5 horas), por lo que el sistema opera mayormente sin sobrecarga.

El generador GCL presenta una distribución más dispersa, con mayor cantidad de clientes esperando entre 1 y 4 horas, lo que indica colas más largas. XORShift tiene una distribución algo más equilibrada, aunque también se observan muchos casos con espera prolongada, pero menos que GCL. Xoshiro, en cambio, tiene mas clientes con espera 0 y muestra la distribución más concentrada hacia la izquierda, con menos casos que superan las 2 horas. Esto confirma lo observado en el gráfico anteriores: Xoshiro tiene menor acumulación de espera

4.4 El porcentaje de tiempo que el servidor está ocupado

Este gráfico muestra el porcentaje total de tiempo que el servidor permaneció ocupado durante las 48 horas de simulación, para cada generador.

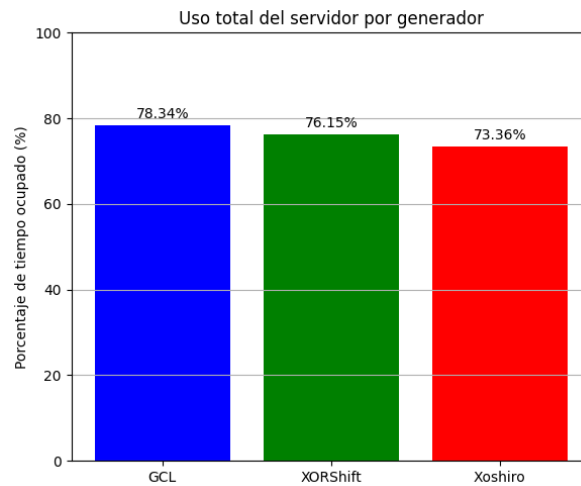


Figure 5: Comparación de porcentajes de ocupación

Si bien las diferencias no son drásticas, se observa que GCL genera una mayor carga sobre el servidor, seguido por XORShift, y finalmente Xoshiro con el menor porcentaje de ocupación. Vemos que menor ocupación no implica ineficiencia, ya que como observamos en los gráficos anteriores, Xoshiro logra tiempos de espera más bajos y menor tiempo promedio en el sistema, manteniendo una carga razonable, lo que puede significar un mejor comportamiento en la simulación de eventos aleatorios.

4.5 Evolución de la longitud de la cola en el tiempo

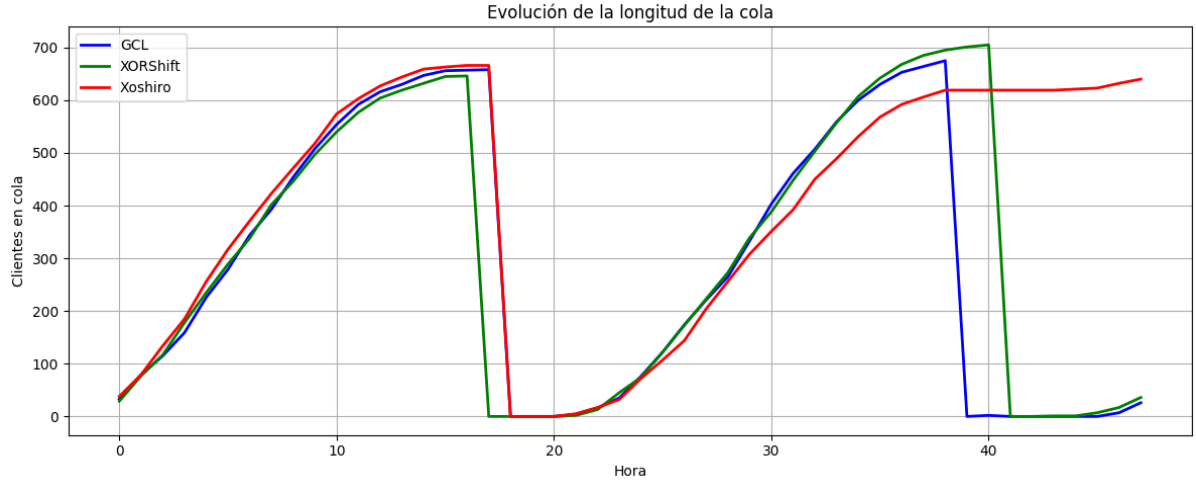


Figure 6: Evolución del largo de la cola con el paso de las horas

En el gráfico podemos apreciar que las tres curvas muestran un comportamiento cíclico, con dos fases de crecimiento que coinciden con los picos de intensidad de arribo definidos por $\lambda(t)$. Posteriormente, en las horas de baja demanda, las colas se vacían casi completamente. Xoshiro, muestra una acumulación más progresiva y una descarga más suave, a comparación de GCL y Xorshift. Esto nos dice que el generador Xoshiro distribuye los eventos mas homogéneamente.

4.6 Histograma de los tiempos de espera en el sistema

Este histograma superpuesto muestra la distribución de los tiempos de espera en el sistema para los tres generadores en una misma escala, lo que permite visualizar con claridad las diferencias en la frecuencia y dispersión de las esperas.

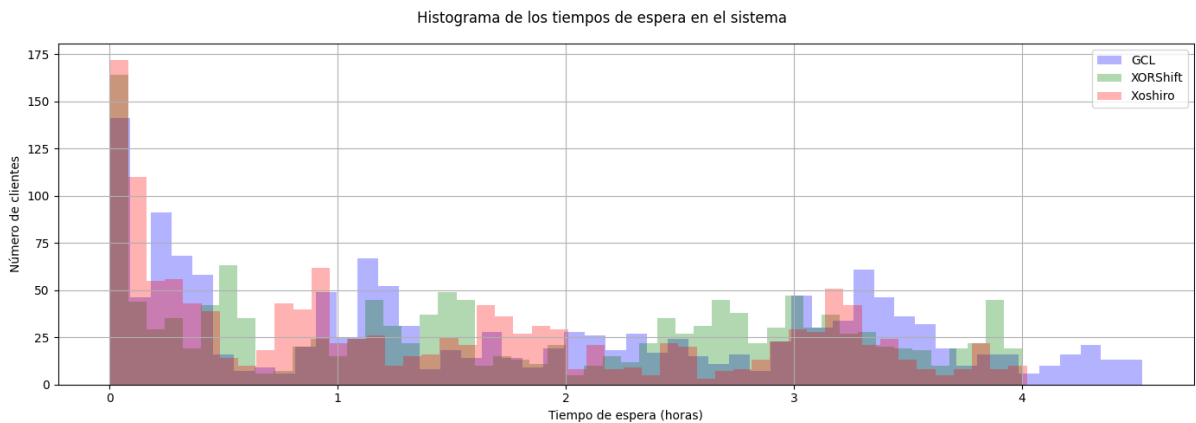


Figure 7: Histogramas con los tiempos de espera

Podemos observar que GCL es el que colas mas largas genera, ya que el numero de clientes con mas de 3hs de espera es mayor al de los otros generadores. También se

aprecia, como vimos antes, que Xoshiro es el que genera mayor cantidad de clientes con poca demora.

4.7 Distribución del tiempo entre arribos y de servicios simulados

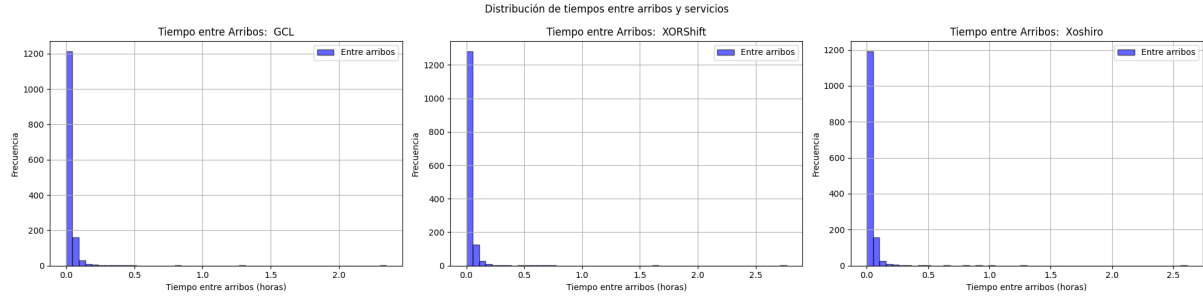


Figure 8: Distribuciones del tiempo entre arribos

En los tres casos, los tiempos entre arribos presentan una distribución fuertemente sesgada a la derecha, con la gran mayoría de los valores concentrados por debajo de los 0.1 horas. Esto es esperable dado que los arribos se generan con un proceso de Poisson no homogéneo y con intensidad $\lambda(t)$, lo que produce una alta frecuencia de llegadas, especialmente en las horas pico.

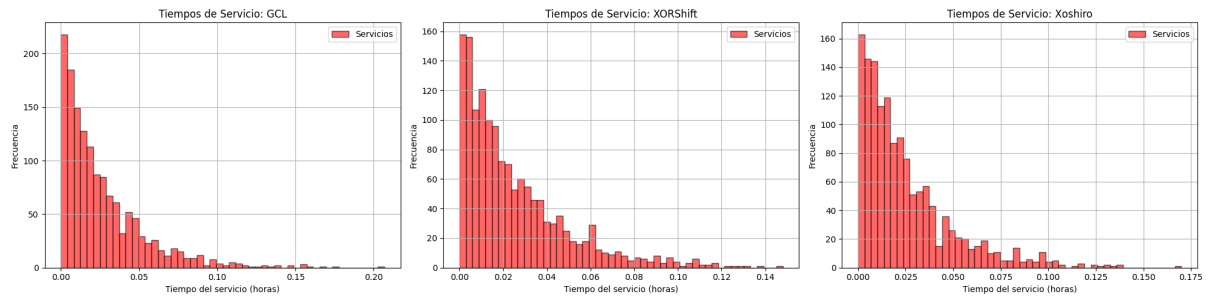


Figure 9: Distribuciones del tiempo de servicio

Los histogramas de tiempos de servicio muestran distribuciones exponenciales, como se espera por el método de generación de atenciones. También aquí se observa que los tres generadores mantienen una forma muy parecida, con valores que se concentran en los primeros 0.02 a 0.04 horas y una caída progresiva hacia la derecha. No se presentan diferencias relevantes entre los generadores en cuanto a la forma.

Tanto para las distribuciones de los tiempos entre arribos como para las de los tiempos de servicio, la forma es muy similar para los 3 generadores. Por lo tanto podemos decir que reproducen correctamente las distribuciones teóricas tanto de los arribos como de los servicios.

5 Conclusiones

5.1 Impacto del generador

A lo largo del análisis, observamos el impacto de los 3 generadores (GCL, Xorshift y Xoshiro) en distintas métricas clave del sistema, como la tasa de utilización del servidor, los tiempos de espera, la evolución de la cola y la distribución de los eventos simulados. A partir de los resultados obtenidos podemos concluir que: Todos los generadores fueron capaces de simular correctamente el sistema, respetando la distribución de arribos y tiempos de servicio esperada.

Xoshiro fue el generador que produjo mejores resultados globales en cuanto al rendimiento del sistema. Mostró menores tiempos de espera, menor tiempo promedio en el sistema y más control en la cola.

GCL, generó mayores niveles de ocupación del servidor y tiempos de espera más dispensos. Esto indica que tiende a generar secuencias que pueden provocar momentos de sobrecarga en el sistema.

XORShift mostró un comportamiento intermedio. Su implementación es eficiente, pero en esta simulación no ofreció ventajas claras frente a Xoshiro.

5.2 Aprendizajes alcanzados

A lo largo del desarrollo de este trabajo práctico pudimos comprender en mayor profundidad el impacto que tiene la elección de un generador de números pseudoaleatorios dentro de una simulación estocástica. Comprobamos que el generador utilizado influye directamente en el comportamiento global del sistema modelado: desde la formación de colas hasta la saturación del servidor y los tiempos de espera de los clientes.

References

- [1] Marsaglia, George, 2003, Random number generators, Journal of Modern Applied Statistical Methods, 2 No. 2.
- [2] Park, Stephen and Miller, Keith, 1988, Random Number Generators: Good Ones Are Hard to Find, Commun. ACM
- [3] Vigna, Sebastiano. "xorshift*/xorshift+ generators and the PRNG shootout". Retrieved 2014-10-25. <https://prng.di.unimi.it/>